

C200 PROGRAMMING ASSIGNMENT №7

Professor M.M. Dalkilic

Computer Science

School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

November 3, 2024

Introduction

Due Date: 11:00 PM, Saturday, November 9, 2024 Please note that both Autograder submission <https://c200.luddy.indiana.edu/> and pushing to GitHub **must be done before the deadline. We do not accept late submissions.** As mentioned in the Inscribe post, we will be checking for cheating. If this results in a grade change, we are compelled by the University to send a formal notification to the Dean.

Instructions

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).
2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.
3. Make sure that the **code does not have infinite loop (that never exits) or an endless recursion (that never completes)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.
4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.
5. Once you are done testing your code, comment out the tests i.e. the code under the `__name__ == "__main__"` section.

6. This homework uses two text files: `family.txt` and `fish_data.txt`. Make sure these files are in the appropriate place to read from. These are both ASCII files.

Problem 1: Distance between a point and line

In a 2D Euclidean plane, the formula for determining the distance between a point and line is:

$$d(\text{point}, \text{line}) = \frac{|ax' + by' + c|}{\sqrt{a^2 + b^2}} \quad (1)$$

$$\text{point} = (x', y') \quad (2)$$

$$\text{line} = (ax + by + c) \text{ for variables } a, b, c \quad (3)$$

The following code:

```
1 line = (4,6,-26)
2 point = (2,-4)
3 print(d(point,line))
```

produces:

```
1 5.824
```

Deliverables for Problem 1

- Complete the function.
- Round the output to three decimal places.

Problem 2: Using Radix Sort

In lecture I provided a less succinct radix sort so we could more easily observe each step. Here I write a simpler, though more succinct, version:

```
1 def radix (lst,digit_index = 0):
2     if lst:
3         cluster = [[] for _ in range(10)]
4         for number in lst:
5             index = int(number[-(digit_index + 1)])
6             cluster[index] += [number]
7
8         sorted, unsorted = [],[]
9         for block in cluster:
10            for number in block:
11                if len(number) > digit_index + 1:
12                    unsorted.append(number)
13                else:
14                    sorted.append(number)
15            return sorted + radix(unsorted, digit_index + 1)
16         else:
17             return []
18
19 data21 = ["101","10","12","1000","99","1","5", '100', '120', '990', '↵
310', '0', '301', '102', '654']
20 print(radix(data21))
```

producing

```
1 ['0', '1', '5', '10', '12', '99', '100', '101', '102', '120', '301', '↵
310', '654', '990', '1000']
```

Using the string type helps a lot. That the output remains strings isn't problematic, since we can easily convert to an integer. Observe that the location in the list is the integer version of the digit we're examining. The `digit_index = 0` means we're starting with the last character of the string. Remember, we can iterate through a string using `-1,-2,-3,...` that starts from the right. These are lines 3-6. Once they've been partitioned, we separated strings of larger length (to be radix sorted again looking at the next left digit) from current length.

We can use radix to sort decimals as well:

```
1 def radix_decimal (lst):
2     pass #calls radix
3
4 data22 = [".301",".101",".20",".1",".12",".654",".99",".31",".309",]
5
```

```
6 print(radix_decimal(data22))
7 d_22 = data22[:, :]
8 d_22.sort()
9 print(d_22)
```

produces:

```
1 ['.1', '.101', '.12', '.2', '.301', '.309', '.31', '.654', '.99']
2 ['.1', '.101', '.12', '.20', '.301', '.309', '.31', '.654', '.99']
```

How can we use radix in its original form? Observe that the decimal maintains order when we remove the decimal and pad zeros to the right. For example, in this data, the longest decimal has three digits; therefore, we add zeros to make the length of all the numbers length three, e.g., $.1 \rightarrow 100$, $.99 \rightarrow 990$, and $.20 \rightarrow 200$. After padding zeros to the right, radix sort handles ordering by character and we reinsert the decimal to return correct numeric values. The ordering can be now done correctly with radix. After sorting we have to prepend the decimal and remove trailing zeros. We may, as a result, produce $.2$ instead of the original $.20$, but this doesn't matter. Lastly, we assume we will never had $.0$.

Programming Problem 2: Maximal Value

- Complete the `radix_decimal()` function that *must* use the `radix` function
- You are not allowed to change the `radix` function
- You must return strings with the decimal point and without trailing zeros
- Tip: To remove the trailing zeros, we can use the `strip()` function that we learned in the FileIO lab. However, you will have to call it differently because the default behavior of `strip()` is to remove the white spaces.

Problem 3: Predator-Prey Model

The Lotka-Volterra predator-prey model characterizes two populations of animals—in this example, we'll assume the predator is a fox and the prey is a rabbit. We examine the total populations of each as they interact. For this, I'll write $r[t]$, $f[t]$ to mean the populations of rabbit and fox respectively at $t = 0, 1, 2, 3, \dots, k$. We're interested in the following, given $r[t]$, $f[t]$ how do we compute $r[t+1]$, $f[t+1]$? The models states that we must include four rates:

- The growth rate of the rabbit population. We'll use b_r for birth of rabbits.
- The death rate of rabbits due to fox predation. We'll use d_r to indicate this value.
- The death rate of foxes, e.g., bad hunting skills, disease, scare food. We'll use d_f for this value.
- Then an odd rate: When foxes catch and consume rabbits, they have litters (kits). So there's a rate of turning rabbits into foxes. We'll call this b_f .

The model is:

$$r[t+1] = [r[t] + r[t]b_r - r[t]f[t]d_r] \quad (4)$$

$$f[t+1] = [f[t] + b_f d_r r[t]f[t] - f[t]d_f] \quad (5)$$

We're using subscripting for the lists. This means given values at locations $r[t]$ and $f[t]$ for rabbits and foxes, we can add new members at locations $r[t+1]$, $f[t+1]$. So, the equations simply make a list of values.

```
1 def rabbit_fox(br,dr,df,bf,rabbit,fox,time_limit):
2     i = 0
3     history_rabbit = []
4     history_fox = []
5     while i < time_limit:
6         history_rabbit.append(rabbit)
7         history_fox.append(fox)
8         #marvelous code
9     return history_rabbit,history_fox
10
11 br = 0.03
12 dr = 0.0004
13 df = 0.25
14 bf = 0.11
15 rabbit = 3000 #initial population size
16 fox = 200 #initial population size
17 time_limit = 2000
18 history_rabbit, history_fox = rabbit_fox(br,dr,df,bf,rabbit,fox, ↵
    time_limit)
```

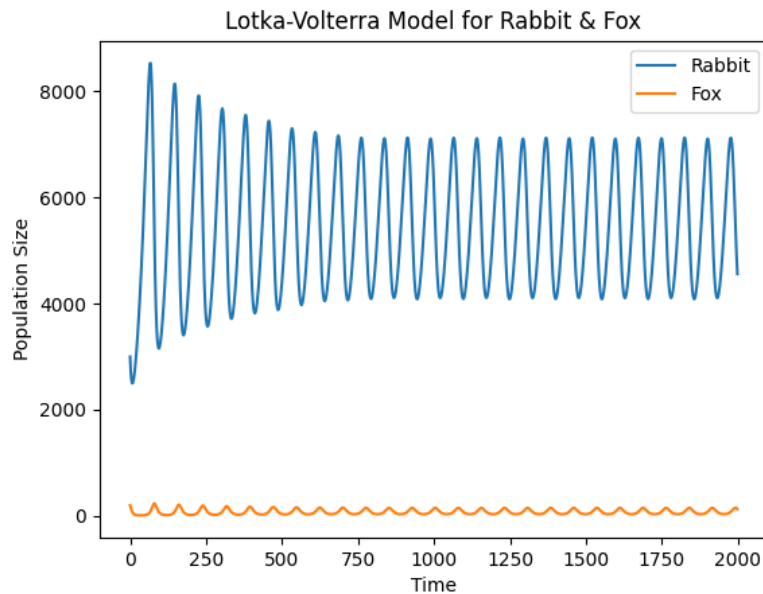


Figure 1: Predator-Prey Lotka-Volterra model for 2000 time points.

produces

```

1 0 3000 200
2 200 5118 18
3 400 4496 150
4 600 6713 42
5 800 4274 46
6 1000 6204 140
7 1200 5902 33
8 1400 4094 77
9 1600 7050 94
10 1800 5132 31

```

which describes the time point, rabbit population, fox population at some points and the graphic in Fig. 1. Your task is to replace the ‘marvelous code’ with the correct code using equations 4 and 5 , and from inspection of the code. You can see the populations reach an equilibrium.

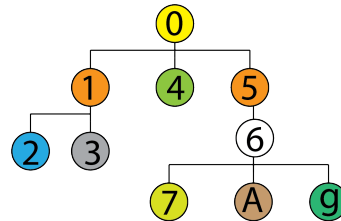
Deliverables for Problem 3

- Observe that equations 4 and 5 use ceiling function.
- Note that we have already given the code for drawing Figure 1 under code tests on line 115-121. After testing the code but before submitting to the Autograder - make sure to comment lines 115-121 (the code used for creating the plot) because Autograder can not interpret visual output i.e. plots and will likely fail due to the code trying to display plots in the web browser. Also, comment out the import of matplotlib before submitting to the Autograder.

Problem 4: Comprehension & File I/O

In this problem, you'll implement functions as comprehensions. The data is in a file called `family.txt` shown in the table and also is given as a visualization. It is a family tree. The names are simply characters, so no type conversion is necessary. You will implement a series of queries based on the data that you read from the file.

Parent	Child
0	1
0	4
0	5
1	2
1	3
5	6
6	7
6	A
6	g



Tree visualizaition of table.

The queries are formed as Python functions. A few are done here:

```
1 #input parent name
2 #output children
3 #constraint using comprehension
4 def get_child(name,data):
5     return [child for parent,child in data if parent == name]
6
7 #input parent name
8 #output true if has children
9 #constraint using comprehension
10 def has_children(name,data):
11     return bool(get_child(name,data))
12
13 #input nothing
14 #output all names
15 #constraint comprehension list only
16 def get_all(data):
17     return [k for k in [i for i,j in data] + [j for i,j in data]]
18
19 print(data4)
20 print(has_children('0',data4)) #true
21 print(has_children('7',data4)) #false
22 print(get_child('6',data4))
23 print(get_parent('g',data4))
24 print(siblings('7','A',data4)) #true
```

```

25 print(siblings('2','7',data4)) #false
26 print(grandparent('0','3',data4)) #true
27 print(grandparent('0','7',data4)) #false
28 print(get_all(data4))
29 print(cousins('3','6',data4)) #true
30 print(cousins('3','5',data4)) #false

```

has outputs:

```

1  [['0', '1'], ['0', '4'], ['0', '5'], ['1', '2'], ['1', '3'], ['5', '6'],
    ], ['6', '7'], ['6', 'A'], ['6', 'g']]
2  True
3  False
4  ['7', 'A', 'g']
5  ['6']
6  True
7  False
8  True
9  False
10 ['0', '0', '0', '1', '1', '5', '6', '6', '6', '1', '4', '5', '2', '3',
    '6', '7', 'A', 'g']
11 True
12 False

```

Programming Problem 4: Comprehension & File I/O

- Complete the code to read the file. Do not change any string—these are the names.
- Complete each of the functions only using comprehension or bool on a comprehension

Problem 5: Max Adjacent

In this problem, you'll implement a function that takes a list of numbers and returns a pair: a sum and a Boolean vector of the numbers that are summed. The condition is that the sum is maximum and that no adjacent numbers can be used. For example,

```
1 data5 = [[5,1,4,1,5],[5,6,2,4],[4,5,1,1],[1,5,10,4,1],[1,1,1,1,1]]
2 for d in data5:
3     print(max_adjacent(d))
```

produces:

```
1 [14, [1, 0, 1, 0, 1]]
2 [10, [0, 1, 0, 1]]
3 [6, [0, 1, 0, 1]]
4 [12, [1, 0, 1, 0, 1]]
5 [3, [1, 0, 1, 0, 1]]
```

In the first list, [5,1,4,1,5] we see that 5+4+5 produces the maximal value; so the Boolean list is [1,0,1,0,1]. Observe no numbers are adjacent. For the second list [5,6,2,4], we could have possibly: 7, [1,0,1,0]; 9,[1,0,0,1], 10,[0,1,0,1]. The output is 10. You are free to implement this as either recursion or looping, but we advise that the recursion is easier if you take advantage of the Boolean list (as you recursively build it) and the base case is a list of length three (no smaller list is used). The length of my solution spans about 30 lines or so, but you might find a simpler solution.

Programming Problem 5: Max Adjacent

- Complete the function.
- You may use either recursion or looping (or both I suppose), but using recursion you can build-up potential sums easily. Observe there are only a few different sums that are possible too.
- You are free to use the sorted function or list.sort too. Using sort you'll be able to restructure the list into *max* sum!

Problem 6: Optimization of Cost

You're contracted to work with a trucking freight company-We Like to Move it, Move it!. The hourly rates for drivers are \$14 /hr and \$15 /hr. The operating cost of the truck is, in cents per mile,

$$\text{operating_cost}(d, s) = \left(\frac{60 + s}{2}\right)d \quad (6)$$

where d, s are distance and speed, respectively. The total cost is then:

$$c(s, d, h) = \text{driver}(h, t) + \text{operating_cost}(d, s) \quad (7)$$

where h is the hourly rate. You inform the company that this is an optimization problem, given a fixed salary and distance. In particular, you say that since the speed must be from 40 mph to 60 mph, there is an optimal speed that reduces cost. Implement the functions that allow you to determine for a fixed distance and salary (we'll look at both), the speed that reduces cost. The following code:

```
1 s_a, s_b = 40, 60
2 hr_rate = 14
3 distance = 100
4 print(min_cost(distance, hr_rate, (s_a, s_b)))
5 hr_rate = 15
6 print(min_cost(distance, hr_rate, (s_a, s_b)))
```

produces:

```
1 (82.91509433962264, 53)
2 (84.77272727272728, 55)
```

One aspect of the problem you must pay attention to is the different units: dollars and cents. These cannot be added together, since they're different units, e.g., 1 dollar plus 1 cent $\neq$ 2. Either you have 100 + 1 or 1 + .01.

Deliverables Problem 6

- Complete the functions **following** the algorithm described above.
- The return type of `min_cost` is a tuple (x, y) where x is the cost and y the speed.

Problem 7: Polynomial Regression using numpy

In this problem, you'll be modeling rock bass otolith data: the size as a function of years. The

Age (year)	Length (inches)
1	4.8
2	8.8
2	8.0
3	7.9
4	11.9
5	14.4
6	14.1
6	15.8
7	15.6
8	17.8
9	18.2
9	17.1
10	18.8
10	19.5
11	18.9
12	21.7
12	21.9
13	23.8
14	26.9
14	25.1

data is in a csv fish_data.txt. When plotted, the data appears polynomial—size three (see Fig. 2).

For this we'll use numpy's polyfit function found here:

<https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html>

You're responsible for reading this document to understand how to use it. To be specific, we have data D and a model

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 \quad (8)$$

From D , we compute the “best” values of a_0, a_1, a_2, a_3 . We saw in the previous problem we can use bounded loops to find these values, but there are a myriad of packages and modules available for us to use. Numpy has a polyfit function that takes the data $D = [(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)]$ as a list of x values and y values and returns an array of a_0, a_1, a_2, a_3 . We can use these with numpy to build a function called poly1d(). In this problem you'll need to:

1. Read the data from the file
2. Read about the package from the url provided and build the graph

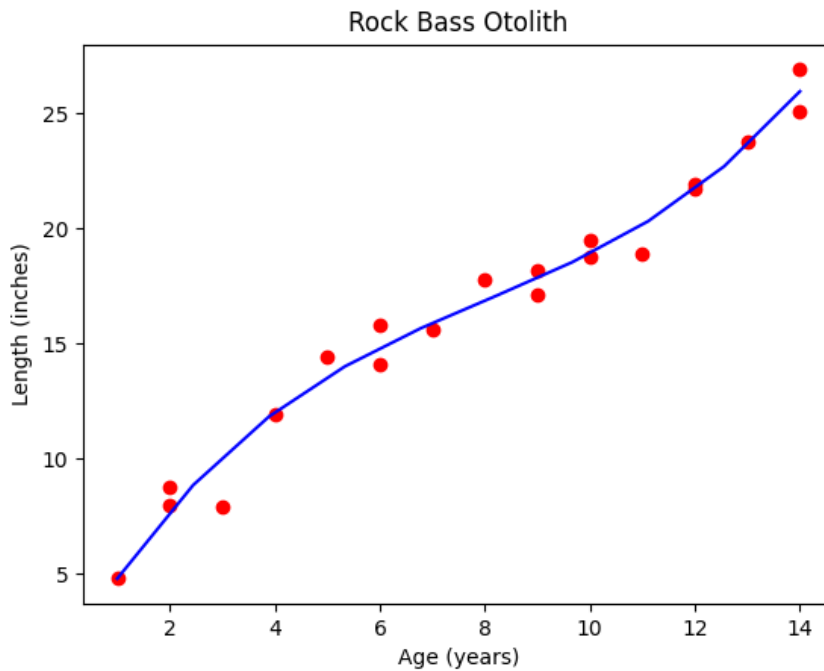


Figure 2: Plots of rock bass data (red) and model (blue). We use numpy's polyfit to do a regression on a polynomial of degree three.

```

1  def get_fish_data(path,name):
2      pass
3
4  #INPUT two lists
5  #RETURN a polynomial function degree 3
6  def make_function(X,Y, degree):
7      pass
8
9  X7, Y7 = get_fish_data(path,name)
10 data4 = [[i,j] for i,j in zip(X7, Y7)]
11 print(data4)
12
13 plt.plot(X7, Y7,'ro')                #plots data D in red
14
15 xp7 = np.linspace(1, 14, 10) #makes input for model
16 p3 = make_function(X7, Y7)  #builds model from data
17 plt.plot(xp7, p3(xp),'b')    #plots model in bule
18
19 plt.xlabel("Age (years)")  #visualization elements
20 plt.ylabel("Length (inches)")
21 plt.title("Rock Bass Otolith")
22 plt.show()

```

gives output:

```
1 [[1, 4.8], [2, 8.8], [2, 8.0], [3, 7.9], [4, 11.9], [5, 14.4], [6, ↵
    14.1], [6, 15.8], [7, 15.6], [8, 17.8], [9,
2 18.2], [9, 17.1], [10, 18.8], [10, 19.5], [11, 18.9], [12, 21.7], [12, ↵
    21.9], [13, 23.8], [14, 26.9], [14, 25.1]]
```

and the plot.

Programming Problem 7: Polynomial Regression with Numpy

- Please read the documentation—please do the small examples shown.
- Complete the functions.
- You should plot the data first (red dots) without plotting the function (blue line) to see if the data is correct.
- Keep in mind that after you have tested your code, you must comment out the 'import matplotlib' and the plotting code given under the `__main__` before submitting to the Autograder. Autograder can not draw graphical plots on the web browser so it will return an error if you do not comment out the plotting code/import matplotlib.
- Unlike the previous file, this has a so-called *header*. Headers are useful, but in the simplest settings can be discarded. Please refer to:
<https://docs.python.org/3/library/csv.html>

8: While Loops and Roots

Remember that for a real-valued function $f(x)$, the root x^* is a value such that $f(x^*) = 0$. We saw, for example, that $ax^2 + bx + c = 0$ has roots that can be analytically determined. Here is a very simple technique to discover roots for $f(x)$ if it is a continuous function.

```
1 #initial estimate
2 x = alpha
3 #epsilon is how good we want the estimate
4 while abs(x - f(x)) > epsilon:
5     x = f(x)
```

While it doesn't work all the time, for some problems, it does astonishingly well for such simple code. The main task is to find the reciprocal of the highest power. For example, assume $f(x) = x^2 - 4x - 7$. The highest power is x^2 so we isolate that:

$$0 = x^2 - 4x - 7 \quad (9)$$

$$x^2 = 4x + 7 \quad (10)$$

$$x = \sqrt{4x + 7} \quad (11)$$

We can quickly see that $f(4) < 0$ and $f(7) > 0$ so a root must lie between. So we start with 4 as our initial guess. The following code:

```
1 def f80(x):
2     return math.exp(-x)
3
4 def f81(x):
5     return math.sqrt(4*x + 7)
6
7 def f82(x):
8     return math.sqrt(math.sqrt(4*x + 7))
9
10 def f83(x):
11     return (3*(x**2)+2)**(1/5)
12
13 x_star = approx_root(f81,4)
14 print(x_star, x_star**2 - 4*x_star - 7)
15 x_star = approx_root(f80,.5)
16 print(x_star, x_star - math.exp(-x_star))
17 x_star = approx_root(f82,4)
18 print(x_star, x_star**4 - 4*x_star - 7)
19 x_star = approx_root(f83,3)
20 print(x_star, x_star**5 - 3*(x_star**2) - 2)
```

produces

```

1 5.3166247040232015 -5.726630121216658e-07
2 0.5671433381054858 7.474599861279074e-08
3 1.9631351074137187 5.407492178477469e-07
4 1.5630377648190852 2.2594686388544005e-06

```

We know one of the roots is $\frac{4+\sqrt{16+28}}{2} \approx 5.3166247903554$ which is very close. We do one more example. Let $f(x) = x - e^{-x}$. Then

$$x = e^{-x} \quad (12)$$

Let's start with an initial guess as $x = .5$. Then

```

1 def f0(x):
2     return math.exp(-x)
3
4 def approx_root(f, initial_guess):
5     ...
6 x_star = approx_root(f0, .5)
7 print(x_star, x_star - math.exp(-x_star))

```

produces

```

1 0.5671433381054858 7.474599861279074e-08

```

The other two functions' roots are:

$$f_2(x) = x^4 - 4x - 7 \quad (13)$$

$$f_3(x) = x^5 - 3x^2 - 2 \quad (14)$$

Deliverables Problem 8

- Complete the functions.
- You can verify your outputs if they're very, very small numbers.
- Use $\epsilon 10^{-7}$

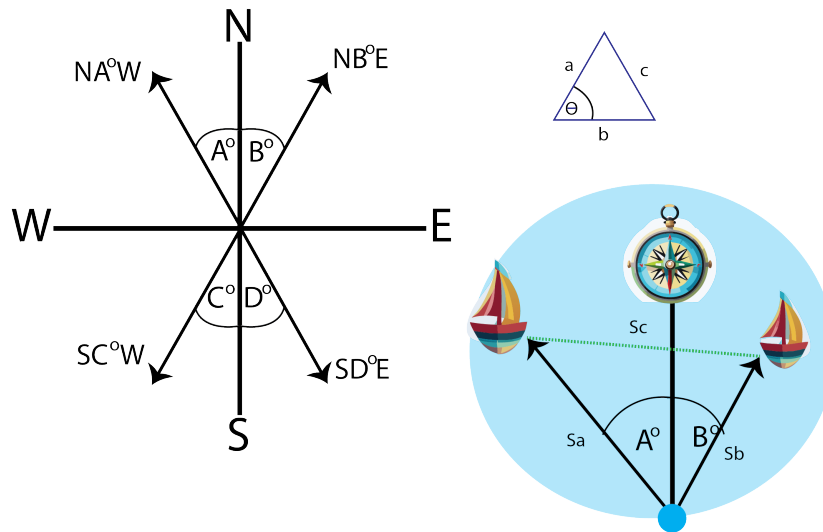


Figure 3: Upper left are the four bearings. Upper right is law of cosines triangle. Lower right are the two ships in our problem. If ship A has traveled S_a miles and B S_b miles, and the angle between is found from their respective bearings, then we can use the law of cosines to find S_c .

9: Sailing Away

In Fig. 3 the use of *bearing* is shown. In shipping, there are four directions used, called bearings, the trajectory of a vessel. Here is the BNF:

$$\langle \text{Bearing} \rangle ::= (\text{N}|\text{S})\langle \text{degree} \rangle(\text{E}|\text{W}) \quad (15)$$

$$\langle \text{degree} \rangle ::= 0.00 | \dots | 360.00 \quad (16)$$

When moving north east, an acute angle is used from north moving clockwise. When moving north west, an acute angle is used from north moving counterclockwise. For south, this is repeated. For this problem, we'll write a general problem for two ships (Fig. 3. lower right). Assume one is headed bearing NX°E at some speed and a second is headed bearing NY°W at some speed. After some time t has elapsed, how far apart are two ships? We can use the law of cosines, where a, b are the distances they've travelled individually and θ the angle between since we can simply sum the two angles from the bearing: For a triangle with the length of two sides a, b known and the angle θ they subtend, we can find the remaining length c as:

$$c^2 = a^2 + b^2 - 2ab \cdot \cos(\theta) \quad (17)$$

The Fig. 3 at the upper right has a triangle labeled.

$$\langle \text{Bearing} \rangle ::= (\text{N}|\text{S})\langle \text{degree} \rangle(\text{E}|\text{W}) \quad (18)$$

$$\langle \text{degree} \rangle ::= 0.00 | \dots | 360.00 \quad (19)$$

You'll be implementing two functions, one for distance and one for the law of cosines. To test this problem:

- At 1:00P, ship A leaves the harbor bearing N34°W at 24 mph.
- At 1:30P, ship B leaves the harbor bearing N56°E at 18 mph.

Find the distance between the two ships, A and B, at 3:00P.

The following code:

```

1 Sa = 24
2 Sb = 18
3 start_a = 1
4 start_b = 1.5
5 stop = 3
6 a,b = 34,56
7 print(cosine_law(*(distance(start_a,stop,Sa),distance(start_b,stop,Sb)↵
    ),a+b))
8
9 s_a,s_b = 40,60
10 hr_rate = 14
11 distance = 100
12 print(min_cost(distance,hr_rate,(s_a,s_b)))
13 hr_rate = 15
14 print(min_cost(distance,hr_rate,(s_a,s_b)))

```

produces

```

1 55.07
2 (82.91509433962264, 53)
3 (84.77272727272728, 55)

```

The `*(distance(start_a,stop,Sa),distance(start_b,stop,Sb))` is sending a tuple of two distance values to the `cosine_law` function.

Deliverables Problem 9

- Complete the functions.

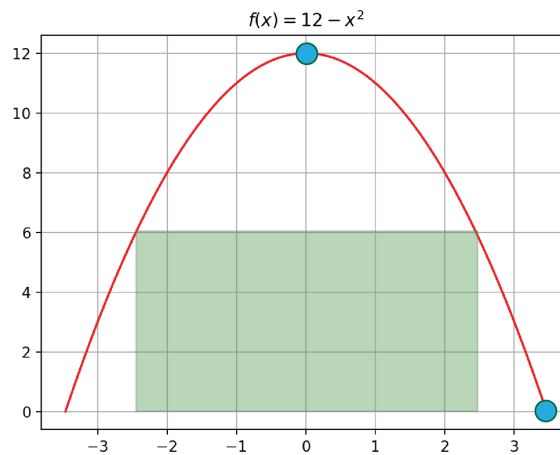


Figure 4: This is **not** to scale. Sketch of what the largest area of a rectangle inscribed $f(x) \geq 0$ for $f(x) = 12 - x^2$.

10: Optimization

Observe Fig. 4. The function highlighted is $f(x) = 12 - x^2$ as the red line. We inscribe a rectangle for $f(x) \geq 0$. Find the dimensions of the largest rectangle possible. Observe that you only need to look at $x \in [0, 2\sqrt{3}]$, but make sure about the dimensions!

The following code:

```

1
2 a,b = 0,2 * math.sqrt(3)
3
4 def f(x):
5     return 12 - x**2
6
7 print(op_rect(a10, b10, f10))

```

produces

```

1 (4.0000000000000003, 7.9999999999999995)

```

Deliverables Problem 10

- Complete the functions
- While we are using half of the interval $[0, 2\sqrt{3}]$ (please pay attention to that)
- *hint: don't use range if you want to save time*

11: Revenue Optimization

A company you've contracted for has the following models for price, revenue, cost, and profit, respectively:

$$p(x) = 5.00 - 0.002x \quad (20)$$

$$R(x) = xp(x) \quad (21)$$

$$C(x) = 3.00 + 1.10x \quad (22)$$

$$P(x) = R(x) - C(s) \quad (23)$$

To find the unit number to maximize P , we *could* take a derivative $\frac{dP}{dx}$ and solve the critical point. On the other hand we can look through the space of solutions. The following code:

```
1
2 p_11, u_11 = max_revenue(R,1,1000)
3 print(u_)
4
5 # to make the graph
6 #x = np.linspace(1,1299,100)
7 #y = P(x)
8 #plt.plot(x,y,'r')
9 #plt.plot(u_,p_,'go')
10 #plt.xlabel("Units")
11 #plt.ylabel("Revenue $MM")
12 #plt.title ("Maximizing Revenue")
13 #plt.show()
```

produces

```
1 975
```

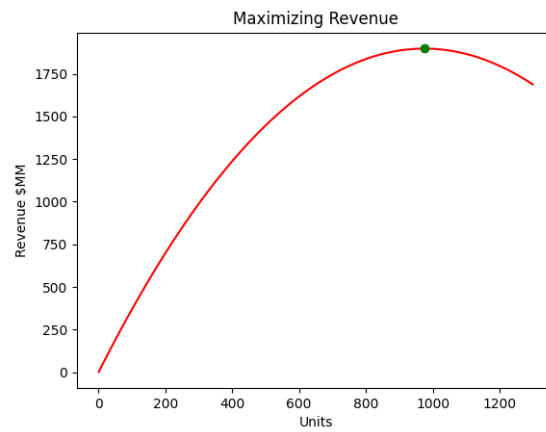


Figure 5: Plotting profit as function of units. The green dot, at 975, represents the maximal profit.

Deliverables Problem 11

- Complete the functions.
- Observe that the function returns a tuple (x, y) where x revenue and y are unit number.
- The visualization code given under code tests should be commented out before submitting to the Autograder.

Problem 12: Same, but Different

This problem will ask you to break the problem into smaller, function parts. When solving a complex computing problem, we never write one monolithic program. We break the problem into small, manageable parts and integrate them. For this problem, you are encouraged to look at previous homeworks too. The task is paradoxically simple. Given a string $s_0s_1\cdots s_m$ (length is $m+1$) and a number n , return a list of the longest string that has, at most, n distinct symbols. Of course, there might be more than one string satisfying this. For example, the code:

```
1 data11 = ["aaaba", "abcba", "abbcd", "aaabbbbaaaac", "abcdeffg"]
2     for d in data11:
3         for i in range(1,7):
4             print(f"{d} with {i} max is\n {max_n(d,i)}")
```

returns:

```
1 aaaba with 1 max is
2   ['aaa']
3 aaaba with 2 max is
4   ['aaaba']
5 aaaba with 3 max is
6   ['', '']
7 aaaba with 4 max is
8   ['', '']
9 aaaba with 5 max is
10  ['', '']
11 aaaba with 6 max is
12  ['', '']
13 abcba with 1 max is
14  ['a', 'b', 'c']
15 abcba with 2 max is
16  ['bcb']
17 abcba with 3 max is
18  ['abcba']
19 abcba with 4 max is
20  ['', '']
21 abcba with 5 max is
22  ['', '']
23 abcba with 6 max is
24  ['', '']
25 abbcde with 1 max is
26  ['bb']
27 abbcde with 2 max is
28  ['abb', 'bbc']
29 abbcde with 3 max is
```

```

30  ['abbc', 'bbcd']
31  abbcde with 4 max is
32  ['abbcd', 'bbcde']
33  abbcde with 5 max is
34  ['abbcde']
35  abbcde with 6 max is
36  ['']
37  aaabbbbaaaaaac with 1 max is
38  ['aaaaaa']
39  aaabbbbaaaaaac with 2 max is
40  ['aaabbbbaaaaaa']
41  aaabbbbaaaaaac with 3 max is
42  ['aaabbbbaaaaaac']
43  aaabbbbaaaaaac with 4 max is
44  ['']
45  aaabbbbaaaaaac with 5 max is
46  ['']
47  aaabbbbaaaaaac with 6 max is
48  ['']
49  abcdeffg with 1 max is
50  ['ff']
51  abcdeffg with 2 max is
52  ['eff', 'ffg']
53  abcdeffg with 3 max is
54  ['deff', 'effg']
55  abcdeffg with 4 max is
56  ['cdeff', 'deffg']
57  abcdeffg with 5 max is
58  ['bcdeff', 'cdeffg']
59  abcdeffg with 6 max is
60  ['abcdeff', 'bcdeffg']

```

For example, let's find the longest strings with at most 2 different symbols:

```

1  aaaba with 2 max is
2  ['aaaba']
3  abbcde with 2 max is
4  ['abb', 'bbc']
5  aaabbbbaaaaaac with 2 max is
6  ['aaabbbbaaaaaa']
7  abcdeffg with 2 max is
8  ['eff', 'ffg']

```

The first string returns the entire string—there are only two symbols entirely. The second string returns the first three 'abbcde'[0:3] and 'abbcde'[1:4]. Including any other symbols would violate

$n = 2$. The second to last string includes every symbol except the last.

The best approach is to break the program into simpler parts. Work with small examples first. You should begin thinking about this at the start of the homework.

Programming Problem 12: Same, but Different

- Complete the function.
- You are strongly encouraged to write several smaller (local) functions to solve this problem. You cannot write global functions, because the autograder can't determine what you need.
- *hint*: Look to other, older homework for some ideas.

Programming partners

ckbarts@iu.edu, seuryu@iu.edu
evbutts@iu.edu, lydiwang@iu.edu
langarbe@iu.edu, olsoncs@iu.edu
laigrant@iu.edu, isvoor@iu.edu
felkhatt@iu.edu, msharsh@iu.edu
mkulekci@iu.edu, marsh1@iu.edu
ssdatla@iu.edu, lukthiel@iu.edu
reclay@iu.edu, ncw1@iu.edu
maboncos@iu.edu, kalerobi@iu.edu
shkami@iu.edu, tanilitt@iu.edu
sydforem@iu.edu, ainlukas@iu.edu
rdotsu@iu.edu, elwesley@iu.edu
keswa@iu.edu, lialaza@iu.edu
mjfarrin@iu.edu, pp22@iu.edu
dajuhnke@iu.edu, tlichten@iu.edu
dgiffor@iu.edu, muksingh@iu.edu
cedrchen@iu.edu, gl25@iu.edu
crigonza@iu.edu, davepais@iu.edu
btbasing@iu.edu, ssp4@iu.edu
jb108@iu.edu, jiajtang@iu.edu
nimjain@iu.edu, jwubelho@iu.edu
concraft@iu.edu, aralover@iu.edu
pchitwoo@iu.edu, audscha@iu.edu
mrb9@iu.edu, corawang@iu.edu
ramgowda@iu.edu, jl367@iu.edu
joglickm@iu.edu, cjrodenb@iu.edu
boydmad@iu.edu, stewrach@iu.edu

iblay@iu.edu, gsackie@iu.edu
eberens@iu.edu, jjrinald@iu.edu
chfurlow@iu.edu, vnathany@iu.edu
natkwon@iu.edu, cpmann@iu.edu
ygajjar@iu.edu, zs15@iu.edu
ealfalah@iu.edu, slopezve@iu.edu
mailic@iu.edu, nmeely@iu.edu
nicdelg@iu.edu, hailreid@iu.edu
ghargro@iu.edu, irodeman@iu.edu
skyangel@iu.edu, nealsing@iu.edu
zahaidar@iu.edu, westje@iu.edu
banksda@iu.edu, sholove@iu.edu
btcunnin@iu.edu, mjseymou@iu.edu
vkabra@iu.edu, prnemani@iu.edu
kadlakha@iu.edu, mitcal@iu.edu
mbenites@iu.edu, jw363@iu.edu
adagar@iu.edu, jvmenach@iu.edu
srkagama@iu.edu, nshnoble@iu.edu
mdevara@iu.edu, tkthang@iu.edu
mugabr@iu.edu, reshetty@iu.edu
blbinder@iu.edu, ozopich@iu.edu
dakomi@iu.edu, adivaidy@iu.edu
addfishe@iu.edu, wl52@iu.edu
pranchan@iu.edu, kzerbino@iu.edu
ldeveau@iu.edu, marcmarq@iu.edu
jdkakare@iu.edu, anangla@iu.edu
salvchri@iu.edu, kpazdur@iu.edu
goldfusl@iu.edu, ansjmeht@iu.edu
adj18@iu.edu, vpottete@iu.edu
agrief@iu.edu, conaitis@iu.edu
herncr@iu.edu, robreese@iu.edu
ag38@iu.edu, cardnorr@iu.edu
tycarmo@iu.edu, manpate@iu.edu
cdasari@iu.edu, bgtruong@iu.edu
clarkina@iu.edu, mszach@iu.edu
jejea@iu.edu, drimawi@iu.edu
vicbarre@iu.edu, matyun@iu.edu
armstan@iu.edu, mawlorch@iu.edu
efoltyn@iu.edu, antonegr@iu.edu
jahjacks@iu.edu, mmalviya@iu.edu
mthinman@iu.edu, risvphil@iu.edu

alurs@iu.edu, kr15@iu.edu
twchase@iu.edu, sasubb@iu.edu
timolaws@iu.edu, amarynow@iu.edu
zalsamri@iu.edu, isapittm@iu.edu
bronburk@iu.edu, pmiesch@iu.edu
cjo3@iu.edu, shwanara@iu.edu
grhamlin@iu.edu, mt87@iu.edu
rycostin@iu.edu, djtheodo@iu.edu
skalluru@iu.edu, jawegman@iu.edu
dh43@iu.edu, nmenne@iu.edu
owencape@iu.edu, kevwong@iu.edu
florech@iu.edu, cmekat@iu.edu
bgelard@iu.edu, bogawa@iu.edu
asgurram@iu.edu, omwells@iu.edu
adhimat@iu.edu, vivitran@iu.edu
heissler@iu.edu, braxluns@iu.edu
talfares@iu.edu, zhangsuo@iu.edu
chjflor@iu.edu, aramanu@iu.edu
jbosio@iu.edu, dyllopez@iu.edu
coubenne@iu.edu, jopantoj@iu.edu
elscheng@iu.edu, tnerkar@iu.edu
nitiarun@iu.edu, lemarc@iu.edu
drussow@iu.edu, fasrasoo@iu.edu
ajkoutsa@iu.edu, ishyadav@iu.edu
ttfender@iu.edu, evanmayo@iu.edu
bahls@iu.edu, ibshaw@iu.edu
natkling@iu.edu, samyadav@iu.edu
ethhanna@iu.edu, cs155@iu.edu
ahsans@iu.edu, thebmill@iu.edu
xmdrew@iu.edu, martibr@iu.edu
bcarpine@iu.edu, dmonroyh@iu.edu
kevgross@iu.edu, angzamora@iu.edu
clickc@iu.edu, kiloscot@iu.edu
ojallow@iu.edu, jacloise@iu.edu
keswar@iu.edu, jskains@iu.edu
idasilva@iu.edu, dematt@iu.edu
okirchen@iu.edu, tunnitha@iu.edu
tborder@iu.edu, mkreddy@iu.edu
jkuk@iu.edu, hsh1@iu.edu
jergalan@iu.edu, klohiya@iu.edu
sergonza@iu.edu, neenpate@iu.edu

jlaytin@iu.edu, mmmatias@iu.edu
akoduru@iu.edu, kalovert@iu.edu
yaljabri@iu.edu, fsafianu@iu.edu
skapure@iu.edu, rnalluri@iu.edu
sabyrap@iu.edu, dommonte@iu.edu
agotanco@iu.edu, rvarrier@iu.edu
aarepal@iu.edu, sunathan@iu.edu
adguhan@iu.edu, alzemlya@iu.edu
ndodoh@iu.edu, jmsequei@iu.edu
hdjour@iu.edu, smitfi@iu.edu
drmiguth@iu.edu, mahonm@iu.edu
carthugh@iu.edu, ndmyer@iu.edu
cgcheane@iu.edu, clushell@iu.edu
jigarnes@iu.edu, rheashah@iu.edu
lddial@iu.edu, sahasrir@iu.edu
ljhorman@iu.edu, ijrussel@iu.edu
ahmealsh@iu.edu, emasseng@iu.edu
egilmour@iu.edu, ozsachs@iu.edu
nwconn@iu.edu, everma@iu.edu
gkreutes@iu.edu, dajshaw@iu.edu
bdgonzal@iu.edu, nguyen17@iu.edu
adodaro@iu.edu, sstickl@iu.edu
abondada@iu.edu, liaobrie@iu.edu
eabilius@iu.edu, adenyu@iu.edu
adekolaw@iu.edu, atsarver@iu.edu
aare@iu.edu, maymat@iu.edu
enkarl@iu.edu, aishravi@iu.edu
nifredri@iu.edu, aposton@iu.edu
tbashee@iu.edu, roldenbu@iu.edu
yc127@iu.edu, ctsvetan@iu.edu
jhl14@iu.edu, youngct@iu.edu
ehchoo@iu.edu, bobbyun@iu.edu
garmenda@iu.edu, ydpatil@iu.edu
xbchrist@iu.edu, jakpatel@iu.edu
jjithesh@iu.edu, zz67@iu.edu
rogoenka@iu.edu, pjminne@iu.edu
nbriere@iu.edu, esspatel@iu.edu
mabana@iu.edu, mzuhair@iu.edu
radutta@iu.edu, rosanghv@iu.edu
rgyasini@iu.edu, anryoun@iu.edu
heoji@iu.edu, soyedele@iu.edu

byersjac@iu.edu, missavin@iu.edu
abrking@iu.edu, sumuth@iu.edu
bthenson@iu.edu, ssanjee@iu.edu
dbarbari@iu.edu, mmaplet@iu.edu
cammlanc@iu.edu, rvasude@iu.edu
evarcarm@iu.edu, smitaa@iu.edu
ethdeatr@iu.edu, aidsteph@iu.edu
meiyhe@iu.edu, kokulski@iu.edu
etferba@iu.edu, joevu@iu.edu
aabdulw@iu.edu, athason@iu.edu
mgoenka@iu.edu, tersacks@iu.edu
kbussel@iu.edu, kenmoses@iu.edu
adatwani@iu.edu, cpmathew@iu.edu
jkittur@iu.edu, vpremku@iu.edu
rbernfel@iu.edu, olesmith@iu.edu
sdondeti@iu.edu, wschrama@iu.edu
ldeel@iu.edu, auskirvi@iu.edu
agahlau@iu.edu, bsati@iu.edu
am280@iu.edu, anupand@iu.edu
jamcham@iu.edu, sxanders@iu.edu
bedani@iu.edu, prmundra@iu.edu
dayinla@iu.edu, adimaran@iu.edu
zkabacin@iu.edu, mjo4@iu.edu
nikdevra@iu.edu, anrajput@iu.edu
nfarzane@iu.edu, isundara@iu.edu
nafike@iu.edu, hshyama@iu.edu
jaeblank@iu.edu, ksabloak@iu.edu
jaccryst@iu.edu, taylbryo@iu.edu
pikhatri@iu.edu, rvenigal@iu.edu
iyersan@iu.edu, aidpride@iu.edu
benchart@iu.edu, camoverm@iu.edu
jackchap@iu.edu, whitedyl@iu.edu
lafly@iu.edu, jakepars@iu.edu
damaatki@iu.edu, rasahu@iu.edu
nantinao@iu.edu, jasreese@iu.edu
mahchaud@iu.edu, cmolaski@iu.edu
blstcoop@iu.edu, roberjuv@iu.edu
alneyadk@iu.edu, hmahapat@iu.edu
mahichan@iu.edu, sandro@iu.edu
michdool@iu.edu, rpolu@iu.edu
ajafowl@iu.edu, bawsung@iu.edu

mfbah@iu.edu, jnimmala@iu.edu
mgbেকেle@iu.edu, aluckhau@iu.edu
smhamrah@iu.edu, jrwaren@iu.edu
rkoston@iu.edu, froy@iu.edu
edeitchl@iu.edu, cjlomax@iu.edu
jagoodal@iu.edu, aamuse@iu.edu
jk314@iu.edu, negia@iu.edu
ebederma@iu.edu, cmellgre@iu.edu
jbriar@iu.edu, navann@iu.edu